

Parte 2: Inteligência em Ação — RAG, Orquestração e Integração Full-Stack

Contexto: Integração de Tecnologias Web e Inteligência Artificial

Foco Pedagógico: Pipeline RAG, Orquestração Assíncrona com n8n, Qualidade de Integração e Entrega Final

1. Visão Geral desta Etapa

Se a Parte 1 foi a fundação da casa — as plantas, os alicerces, as paredes estruturais —, esta etapa é onde a casa ganha vida. Você conectarão as camadas já projetadas a um pipeline de inteligência real: documentos que entram, são compreendidos e passam a informar as respostas do assistente.

Mais do que implementar funcionalidades, o desafio aqui é manter a disciplina arquitetural conquistada na etapa anterior diante de uma complexidade crescente. É fácil preservar o isolamento de domínio quando o sistema é simples. O teste real vem quando surgem novos componentes — e a tentação de criar atalhos se torna maior.

O paradigma SDD e o padrão CRISP continuam sendo os instrumentos norteadores. Antes de qualquer linha de código, a especificação dos contratos entre as novas camadas deve estar validada pela equipe.

2. Fundamentos Conceituais do RAG

RAG (Retrieval-Augmented Generation) é a técnica que resolve uma limitação estrutural dos modelos de linguagem: eles são treinados em um ponto fixo no tempo e não têm acesso ao conhecimento específico da sua organização ou domínio.

A solução não é retreinar o modelo — isso seria proibitivo em custo e tempo. A solução é enriquecer a pergunta do usuário com fragmentos de conhecimento relevante recuperados de uma base de dados própria, antes de enviá-la ao modelo. O modelo, então, responde baseado tanto em seu conhecimento geral quanto no contexto específico fornecido.

O problema que o RAG resolve, em termos de sistema:

Sem RAG, o fluxo é: Usuário → Pergunta → LLM → Resposta.

Com RAG, o fluxo é: Usuário → Pergunta → [Busca por contexto relevante] → Pergunta enriquecida → LLM → Resposta fundamentada.

O pipeline RAG tem duas fases completamente distintas, com responsabilidades e momentos de execução diferentes:

- **Fase de Ingestão (Ingestion):** ocorre quando um documento é carregado. É o processo de preparar o conhecimento para ser recuperado eficientemente no futuro.

- **Fase de Recuperação (Retrieval):** ocorre em tempo real, a cada pergunta do usuário. É o processo de encontrar os fragmentos mais relevantes para a pergunta atual.

Essa separação temporal não é apenas um detalhe de implementação — ela é uma decisão arquitetural. Confundir as duas fases ou misturar suas responsabilidades em um único serviço cria um sistema frágil, difícil de otimizar e de testar.

3. Arquitetura do Pipeline de Ingestão

3.1 Separação de Responsabilidades dentro da Ingestão

O DocumentIngestionService pode parecer, à primeira vista, um serviço monolítico que "processa documentos". Essa visão é equivocada e perigosa. Internamente, ele orquestra três responsabilidades completamente distintas, cada uma candidata à sua própria abstração:

Parsing (leitura e extração): a responsabilidade de entender o formato do arquivo — PDF, TXT, DOCX — e extrair o texto puro. O parser não sabe nada sobre o que será feito com esse texto. Ele recebe bytes e devolve string.

Chunking (fragmentação): a responsabilidade de dividir o texto em pedaços com tamanho e sobreposição (*overlap*) adequados para busca semântica. O chunker não sabe de onde veio o texto nem para onde os chunks irão. Ele recebe uma string longa e devolve uma lista de strings menores.

Embedding (vetorização): a responsabilidade de converter cada chunk em um vetor numérico que representa seu significado semântico. O EmbeddingService não sabe nada sobre documentos, chunks ou banco de dados. Ele recebe texto e devolve um vetor de floats.

3.2 O Contrato de Saída da Ingestão

Ao final do pipeline de ingestão, dois contratos devem ser cumpridos:

1. **Persistência:** cada chunk, com seu embedding e metadados de origem, deve estar salvo no `document_chunks` do banco de dados, pronto para ser consultado.
2. **Notificação:** o n8n deve ser notificado via webhook de que o documento foi indexado com sucesso. Esse webhook é um **contrato de saída** do backend — o backend conhece a URL do n8n e dispara a notificação, mas não conhece nem controla o que o n8n fará com ela.

4. Arquitetura do Retrieval Service

A fase de recuperação tem uma responsabilidade única e bem definida: dada uma pergunta do usuário, encontrar os fragmentos de conhecimento mais relevantes armazenados no

banco de dados.

4.1 O RagService como Orquestrador Puro

O RagService não acessa o banco diretamente nem chama o modelo de embedding diretamente. Ele orquestra:

1. Delega ao EmbeddingService a vetorização da pergunta
2. Delega ao DocumentChunkRepository a busca por similaridade vetorial
3. Recebe os chunks recuperados e os formata como contexto para o prompt

Essa cadeia de delegações mantém o RagService testável de forma isolada: é possível verificar seu comportamento sem um banco de dados real e sem chamar a API de embedding.

4.2 A Query de Similaridade e seus Parâmetros de Qualidade

A busca por similaridade no pgvector envolve dois parâmetros críticos que impactam diretamente a qualidade das respostas:

TOP_K (quantos chunks recuperar): valores muito baixos podem perder contexto relevante; valores muito altos inflam o prompt e adicionam ruído.

MIN_SIMILARITY (limiar mínimo de relevância): um limiar muito baixo recupera chunks irrelevantes que confundem o modelo; um limiar muito alto pode não recuperar nada, fazendo o assistente responder sem contexto.

 **Nota Técnica:** O operador `<=>` no pgvector calcula distância de cosseno entre vetores. O score de similaridade é $1 - \text{distância}$: um score de 1.0 indica vetores idênticos; scores abaixo de 0.7 geralmente indicam baixa relevância semântica. Esses limiares são configuráveis e devem ser tratados como parâmetros de negócio, não constantes hardcoded no código.

5. Arquitetura da Interface React para RAG

A interface de chat na Parte 2 evolui além do simples envio e recebimento de mensagens. Ela deve agora comunicar ao usuário a **rastreabilidade** das respostas: quais documentos e trechos específicos foram usados pelo assistente.

5.1 Novos Contratos de Dado no Frontend

O objeto de resposta do backend agora carrega não apenas a mensagem, mas também a lista de fontes utilizadas (sources). Essa adição muda o contrato entre front e back-end, e deve ser refletida nos DTOs e nos tipos TypeScript da camada de API do frontend antes de qualquer implementação visual.

5.2 Separação de Responsabilidades no Frontend com RAG

A gestão das fontes RAG segue os mesmos princípios da Parte 1:

useChatConversation (hook): responsável por capturar o array de sources retornado pela API e disponibilizá-lo como estado. Não decide como as fontes são exibidas.

<MessageBubble /> (componente): responsável por renderizar as fontes colapsáveis abaixo de cada resposta do assistente. Não sabe como as fontes foram obtidas.

<SourcePanel /> (componente): responsável por exibir, em detalhes, os chunks utilizados para auditar a resposta. Recebe os dados como props — não os busca diretamente.

6. Orquestração Assíncrona com n8n

6.1 Posicionamento Arquitetural do n8n

O n8n não é parte do backend. Não é um serviço auxiliar do Spring Boot. Ele é uma **camada de orquestração externa**, completamente independente do domínio da aplicação. Essa distinção tem consequências práticas: o backend expõe contratos claros (endpoints de webhook, endpoints de status, endpoints de callback) e o n8n os consome. A dependência é unidirecional — o n8n conhece o backend; o backend não conhece o n8n.

6.2 Os Dois Contratos que o Backend deve Expor para o n8n

Para que o n8n opere de forma autônoma, o backend deve garantir dois contratos estáveis:

Endpoint de status do documento (GET /api/documents/{id}/status): o n8n precisa verificar se a indexação foi concluída com sucesso, em falha, ou ainda está em andamento. Esse endpoint deve retornar um estado claro e enumerado — não um payload complexo com lógica de interpretação.

Endpoint de health (GET /api/health): já definido na Parte 1 como requisito mínimo de monitoramento, é também o ponto de verificação que o n8n usa para confirmar que o backend está operacional antes de disparar workflows críticos.

6.3 Responsabilidade dos Workflows

Os workflows do n8n existem para executar lógica que não pertence ao domínio da aplicação: envio de notificações, integração com serviços externos (e-mail, Telegram), geração de relatórios periódicos. São tarefas de infraestrutura e comunicação — não regras de negócio.

 **Nota Técnica:** Um workflow do n8n que começa a conter lógica de negócio complexa (validações, cálculos, decisões sobre o estado do domínio) é um sinal de que essa lógica deveria estar no backend. O n8n é para orquestrar; o Spring Boot é para decidir.

7. Qualidade de Integração Full-Stack

7.1 Como Validar que as Quatro Camadas se Comunicam

A integração full-stack não é verificada executando o sistema e "vendo se funciona". Ela é verificada testando cada **contrato** de forma isolada, antes de testá-los em conjunto.

O processo recomendado segue esta sequência:

Etapa 1 — Validação de contratos de API (backend isolado): utilize uma ferramenta de requisição HTTP (Postman, Insomnia, curl) para verificar cada endpoint do backend de forma independente. Confirme que os status codes, os shapes de resposta e os comportamentos de erro estão exatamente conforme especificados.

Etapa 2 — Validação do pipeline RAG (backend + pgvector): faça o upload de um documento de teste com conteúdo conhecido. Em seguida, faça uma pergunta cuja resposta esteja explicitamente no documento. Confirme que a resposta do chat referencia a informação do documento e que as fontes retornadas apontam para o chunk correto.

Etapa 3 — Validação da interface (frontend + backend): confirme que os componentes visuais refletem corretamente os estados da API: loading, resposta com fontes, resposta sem fontes (pergunta fora do escopo dos documentos), e erro.

Etapa 4 — Validação da orquestração (n8n + backend): após o upload de um documento, confirme que o webhook do n8n é disparado, que o workflow executa com sucesso e que a notificação é entregue. Inspeccione os logs de execução do n8n para confirmar que cada nó processou os dados esperados.

7.2 Tratamento de Erros como Contrato, não como Detalhe

O tratamento de erros não é uma funcionalidade adicional — é parte do contrato da API.

Um endpoint que retorna status 200 com uma mensagem de erro no body está quebrando o contrato HTTP. Um endpoint que retorna 500 para um erro de validação que deveria ser 400 está transferindo a responsabilidade de interpretação para o cliente.

Os status codes mínimos que a API deve implementar com semântica correta:

- 400 Bad Request — dados de entrada inválidos ou ausentes
- 404 Not Found — recurso solicitado não existe
- 409 Conflict — conflito de estado (documento já indexado, conversa já encerrada)
- 500 Internal Server Error — falha não recuperável no servidor

8. CRISP Aplicado à Parte 2 — Templates Específicos

Para manter a consistência com o fluxo SDD estabelecido na Parte 1, os prompts desta etapa seguem o mesmo padrão CRISP, com escopo específico para as novas camadas.

Template para Geração do Pipeline de Ingestão:

Aja como um Arquiteto de Software Sênior especialista em Java Spring Boot e sistemas de IA com RAG.

CONTEXTO:

Estamos desenvolvendo o módulo de ingestão de documentos de um assistente inteligente. O sistema já possui a estrutura base de controllers, services e repositories definida na Parte 1. Agora, precisamos implementar o pipeline de ingestão: parsing → chunking → embedding → persistência no pgvector.

INTENÇÃO:

Crie o Documento de Especificação (System Docs) para o DocumentIngestionService e o EmbeddingService. Não escreva o código final. Mapeie as responsabilidades de cada classe, os parâmetros de entrada e saída de cada método público, e o contrato de notificação para o n8n ao final da ingestão.

RESTRIÇÕES INEGOCIÁVEIS:

- 1. O EmbeddingService deve ser agnóstico ao domínio de documentos.*
- 2. O DocumentIngestionService não pode chamar o LLM de geração — apenas o modelo de embedding.*
- 3. A notificação ao n8n deve ser feita via HTTP webhook, sem acoplamento à implementação interna do workflow.*
- 4. Nenhuma lógica de parsing deve estar no controller.*

PARÂMETROS DE SAÍDA:

Markdown contendo: diagrama de sequência textual do fluxo de ingestão, assinaturas dos métodos públicos de cada serviço, e contrato do payload do webhook enviado ao n8n.

OBS: Após validar este System Docs com a equipe, utilize o segundo prompt de execução conforme descrito na Parte 1: "A especificação está aprovada. Baseado APENAS neste documento, gere agora o código para o [Serviço X]."